HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

TOÁN RỜI RẠC

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

DISCRETE MATHEMATIC

Minimum Spanning Tree

ONE LOVE. ONE FUTURE.

Part 1: Combinatorial theory

Part 2: Graph theory

Content: part 2

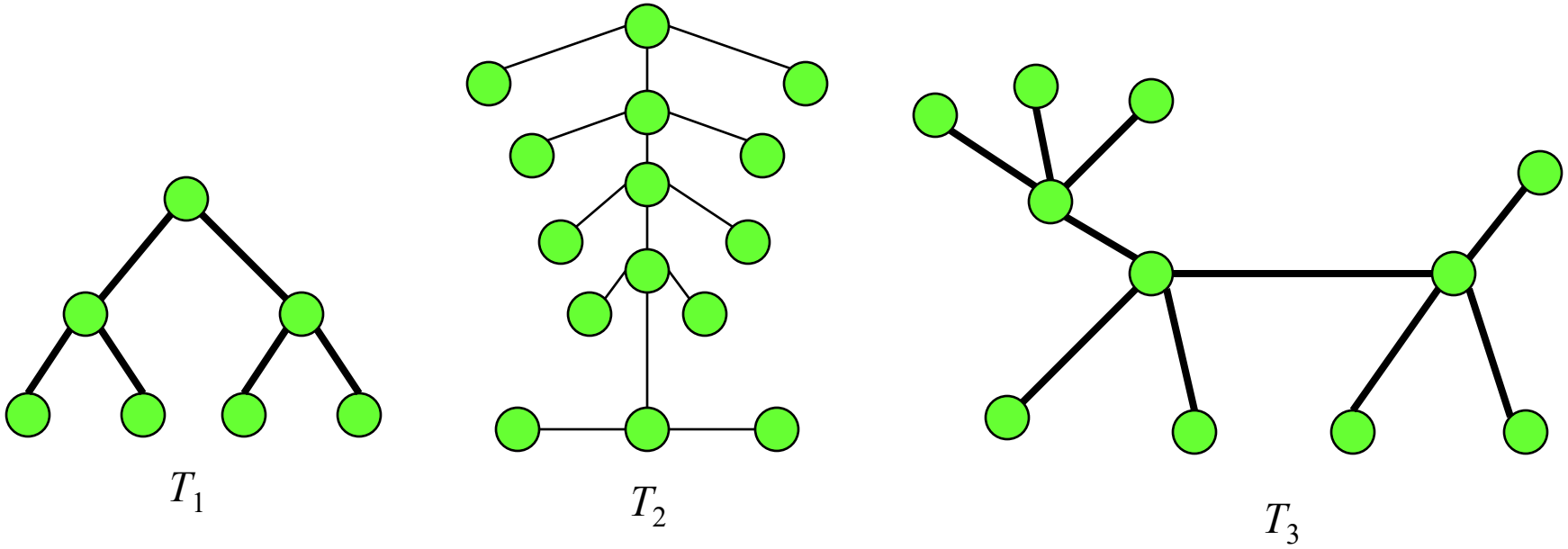
- Chapter 1. Definitions
- Chapter 2. Graph representation
- Chapter 3. Graph traversal
- **Chapter 4. Tree and Minimum Spanning Tree Problem**
- Chapter 5. Shortest path problem
- Chapter 6. Max-Flow

Chapter 4. Tree and Minimum Spanning Tree problem

- Definitions
- Spanning Tree
- Minimum Spanning Tree Problem

Definitions

- Tree is an undirected, connected graph, and contains no cycle
- Forest is an undirected graph containing no cycle
- Each connected component of a forest is a tree



Forest *F* has 3 trees T_1, T_2, T_3

Definitions

Theorem. Given an undirected graph $G = (V,E)$, the following conditions are equivalent:

(1) G is a connected graph with no cycles. (Thus G is a tree by the above definition).

(2) For every two vertices $u, v \in V$, there exists exactly one simple path from u to v.

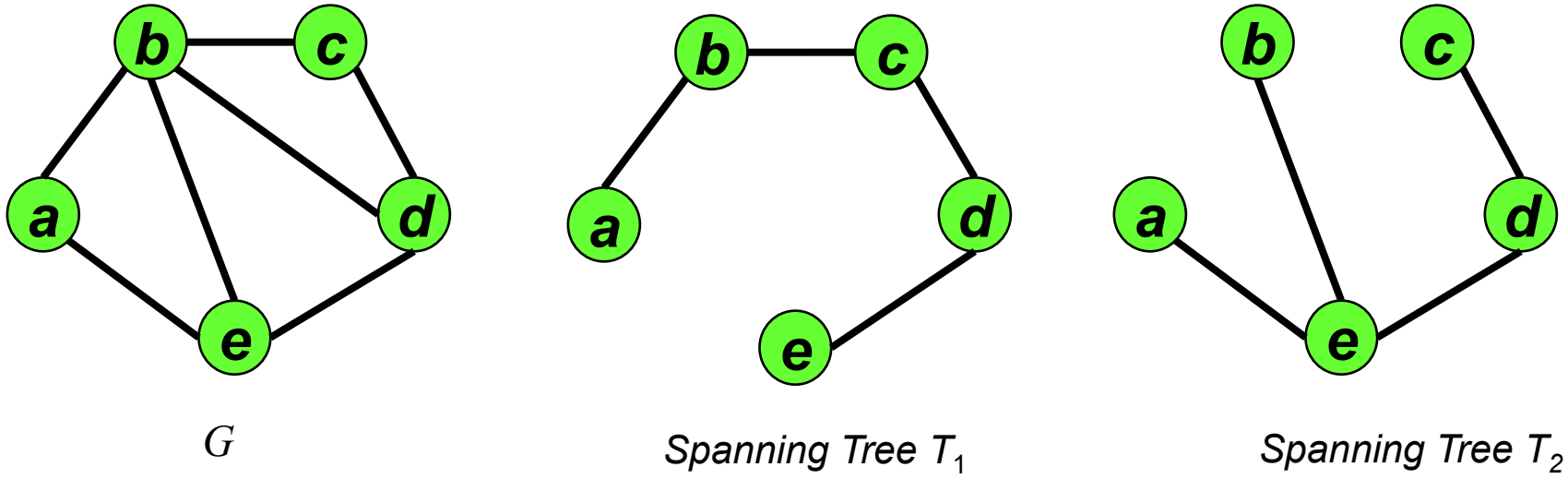
(3) G is connected, and removing any edge from G disconnects it (each edge of G is a bridge).

(4) G has no cycles, and adding any edge to G gives rise to a cycle. (Thus G is a maximal acyclic graph).

(5) G is connected and $|E| = |V| - 1$.

Spanning Trees

- Definition. *Let G=(V,E) is an undirected connected graph. A Tree T=(V,F) having F⊆ E called a spanning tree of G.*

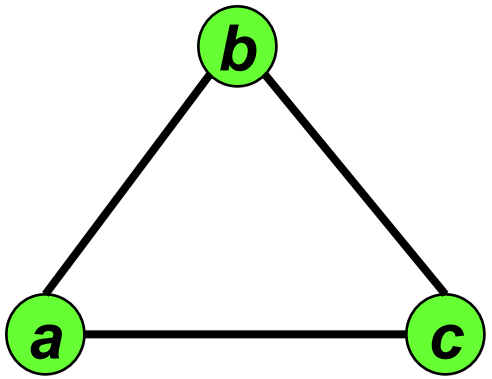


G *Spanning Tree T_1* *Spanning Tree T_2*

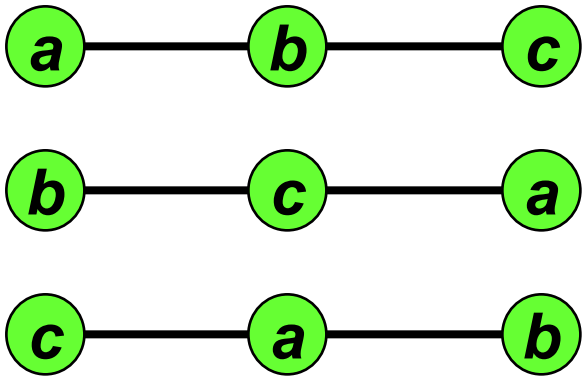
Graph G and 2 spanning trees T_1 and T_2

Spanning trees

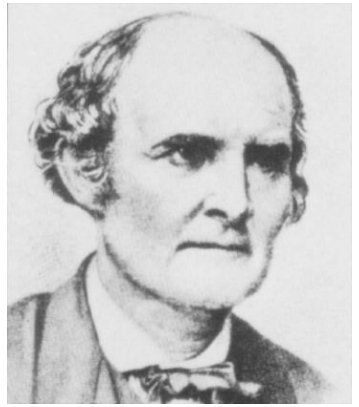
- **Theorem 2 (Cayley).** *Number of spanning trees of* K_n *is* n^{n-2}



K_3



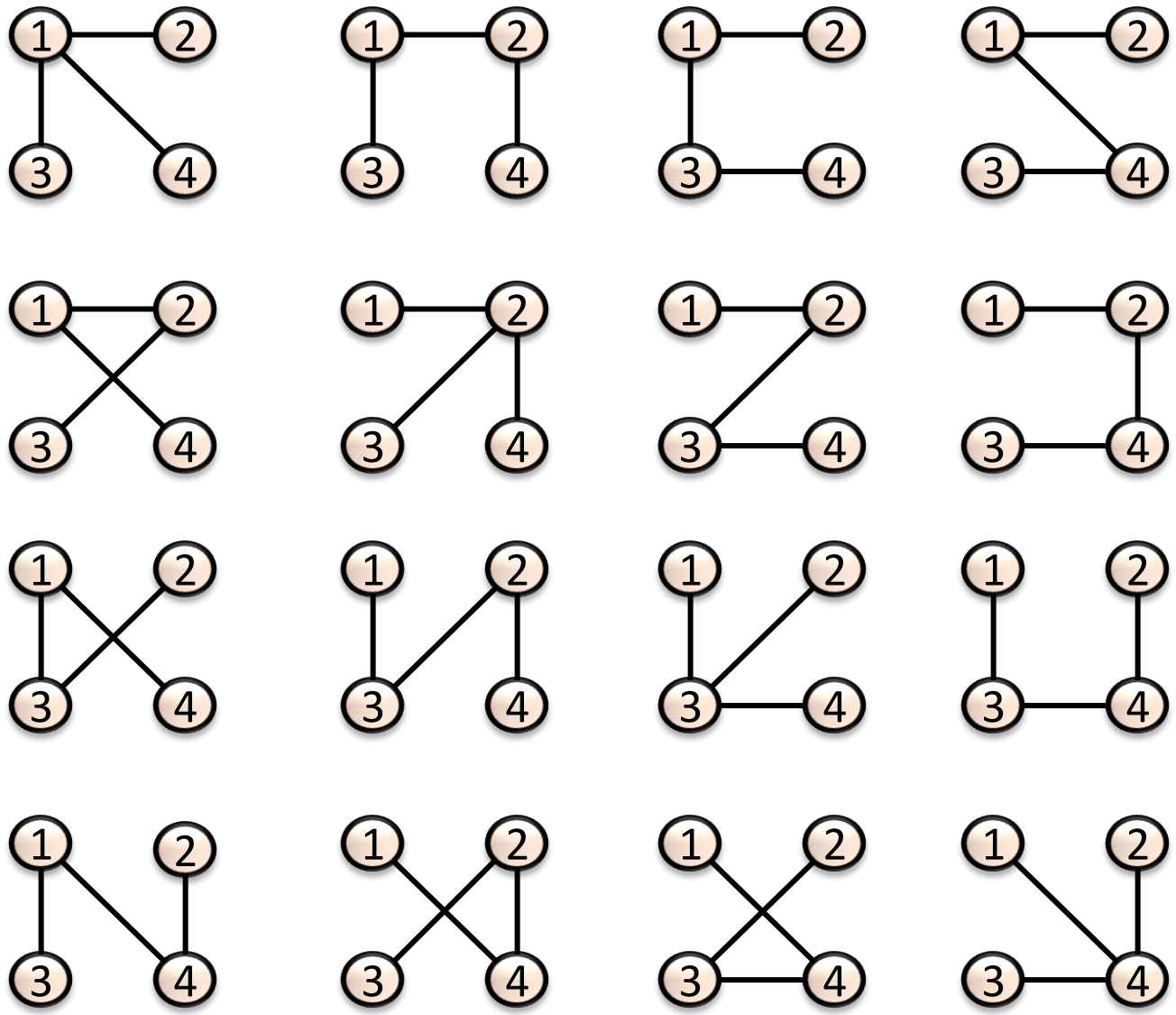
3 spanning trees of K_3



Arthur Cayley
(1821 – 1895)

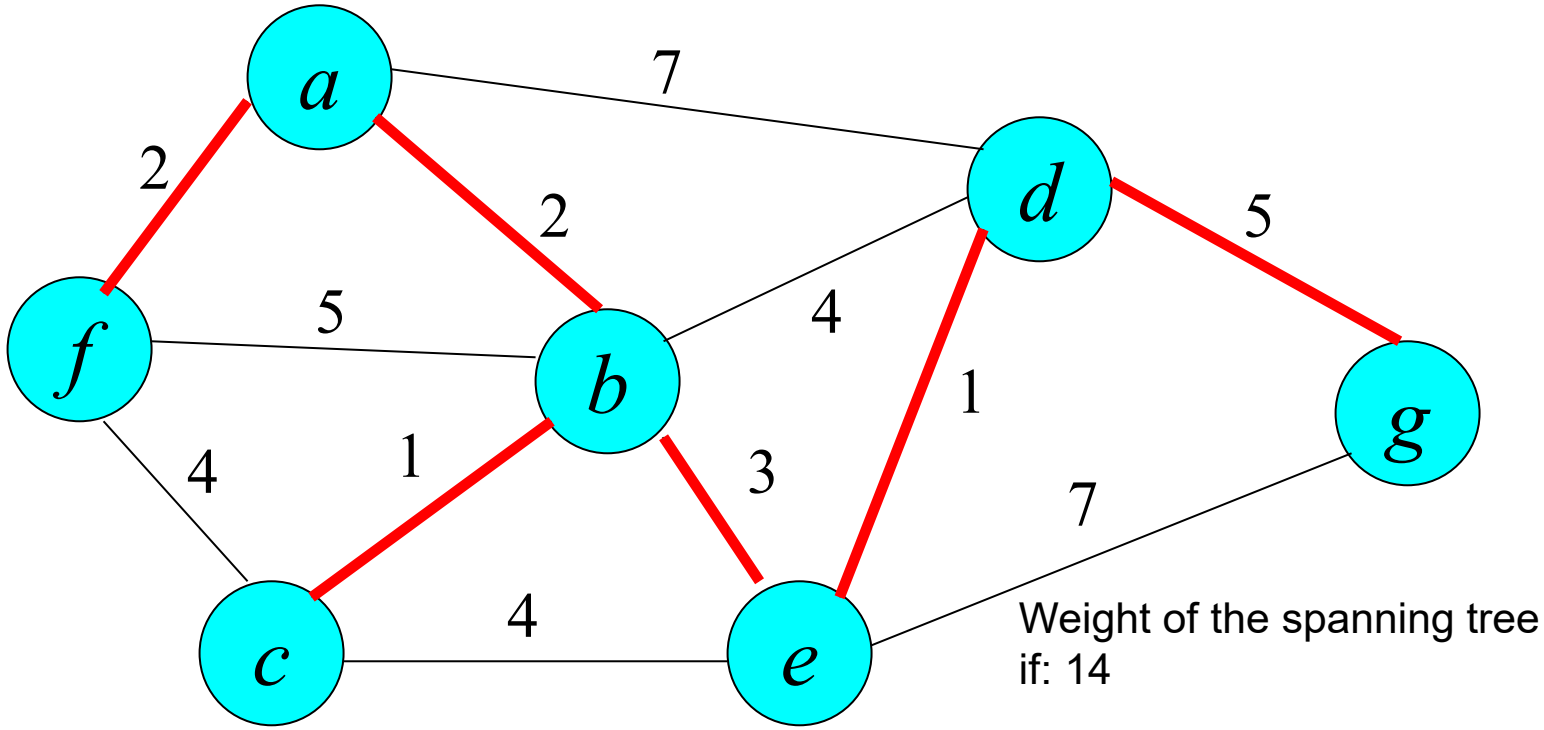
Spanning trees

- **Theorem 2 (Cayley).** *Number of spanning trees of* K_n *is* n^{n-2}



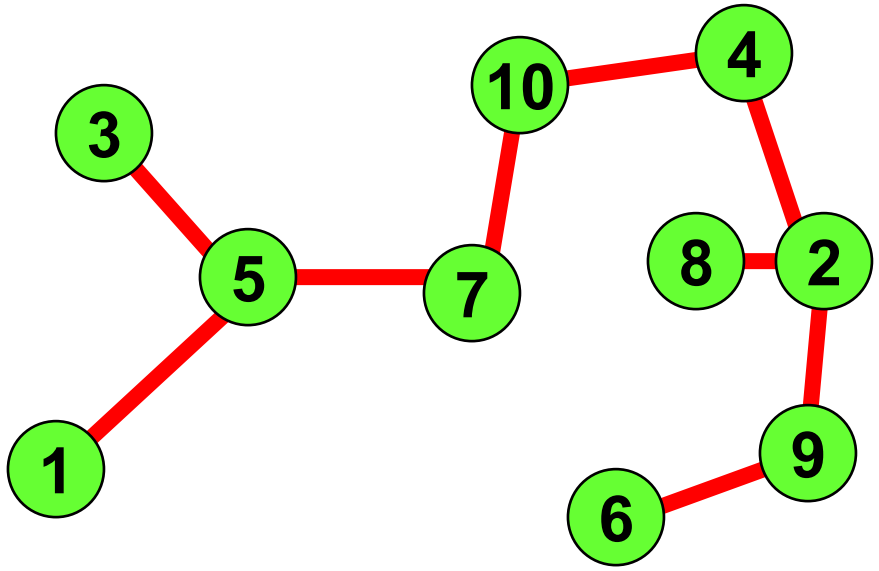
Minimum Spanning Tree problem

- **Problem:** Given an undirected connected graph $G=(V,E)$ in which $c(e)$ is the weight of edge e ($e \in E$). The weight of a tree is the sum of weights of its edges. Find the spanning tree of G having the smallest weight



Minimum Spanning Tree problem (applications)

- Company AT&T want to setup a network connecting *n* customers. Connection cost between *i* and *j* is c_{ij}. Find the network having the smallest weight?



Minimum Spanning Tree problem (applications)

Railway construction

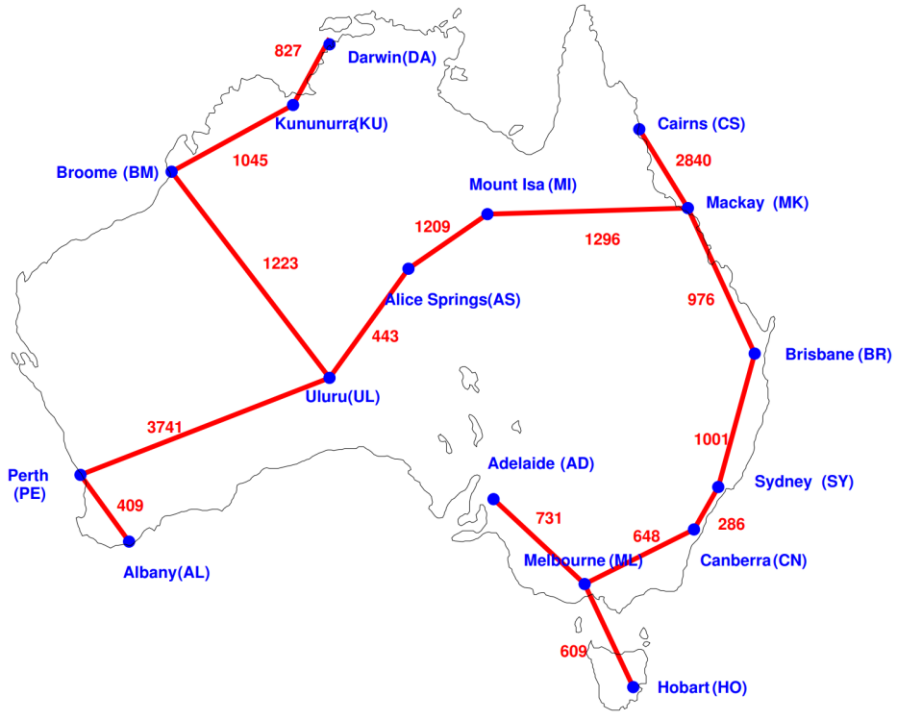
- Suppose we want to build a railway system that connects *n* cities so that passengers can travel between any two cities and the total cost of construction must be minimal.

- It is clear that it corresponds to a graph where vertices are the cities and the edges are the railroads connecting the cities, the length on each edge is the cost of building a railway connecting the two cities.

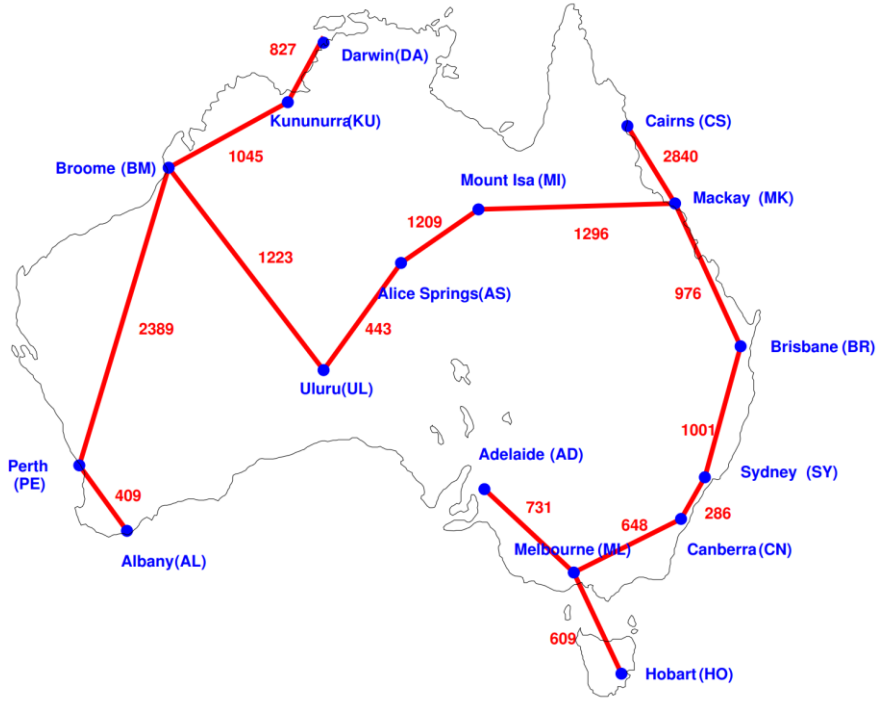
- Therefore, it leads to the problem of finding the smallest spanning tree on a complete graph K_n

Minimum Spanning Tree problem (applications)

Railway construction



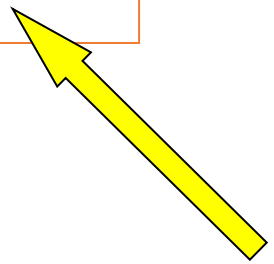
Length: 17284



Length: 15932

Generic Algorithm Schema for Minimum Spanning Tree problem

```
Generic-MST(G, c)
    A = ∅
    //Invariant: A subset of edges of some MST
    while A is not a spanning tree do
            select a safe edge (u, v) of A
            A = A ∪{(u, v)}
            // A is still a subset of edges of some MST
    return A
```



Find the smallest weight edge for inserting into A, that does not create a cycle

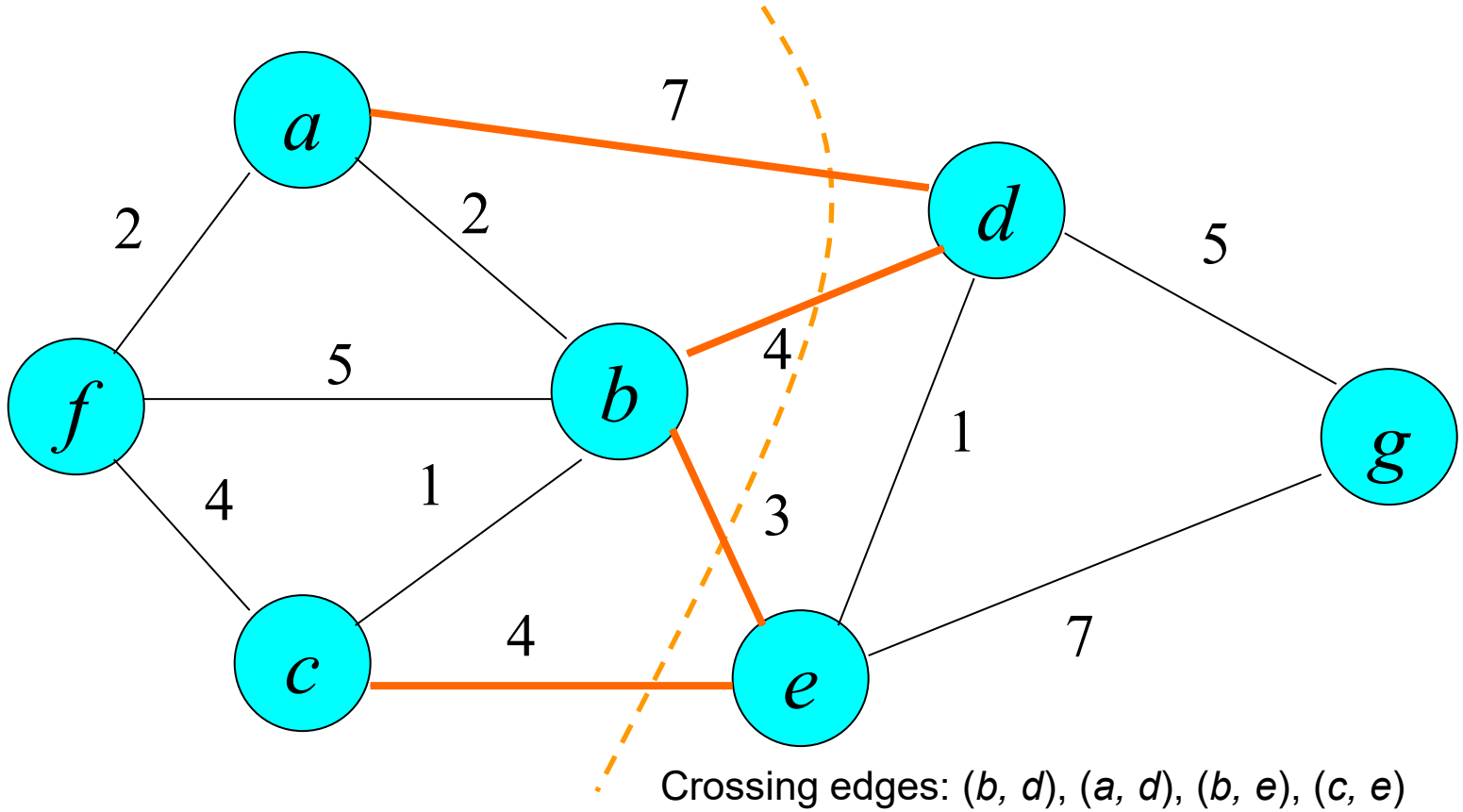
How to find a safe edge?

Cuts

- A cut $(S, V - S)$ is a partition of V into 2 subset S and $V - S$. An edge e is called a crossing edge $(S, V - S)$ if one endpoint belongs to S and the other belongs to $V - S$.

Cuts

- **A cut** of $G = (V, E)$ is a partition of V into $(S, V - S)$.
- **Example** A cut $(S, V\text{-}S)$: $S = \{a, b, c, f\}$, $V - S = \{e, d, g\}$



Cuts

- A cut $(S, V-S)$ is a partition of V into 2 subset S and $V-S$. An edge e is called a crossing edge $(S, V-S)$ if one endpoint belongs to S and the other belongs to $V-S$.
- Suppose A is a subset of edges of G. A cut $(S, V-S)$ is called *compatible* with A if no edge of A is a crossing edge of the cut

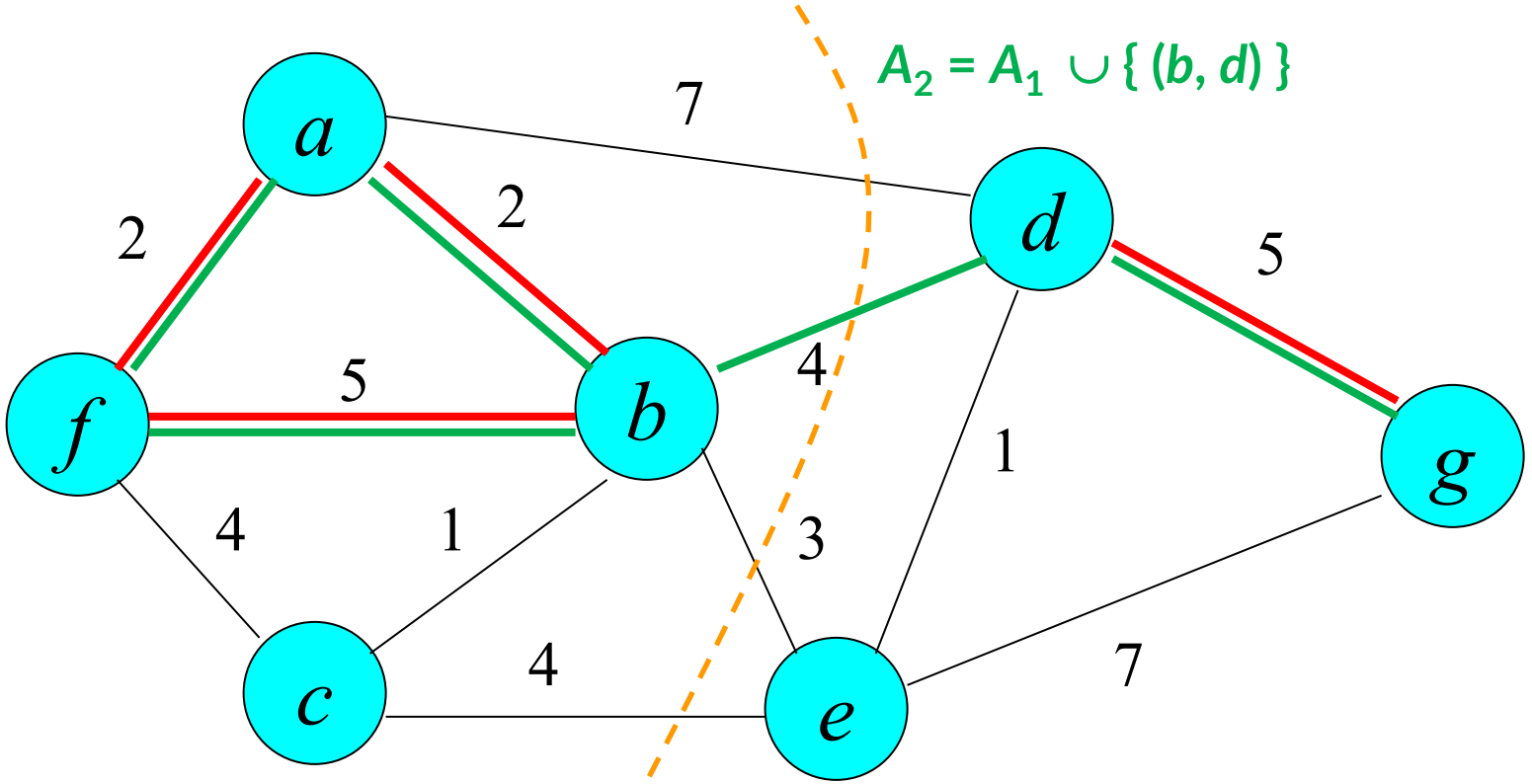
Cuts

Example. Cut $(S, V\text{-}S)$: $S = \{a, b, c, f\}$

Edges:

$A_1 = \{ (a, b), (d, g), (f, b), (a, f) \}$

$A_2 = A_1 \cup \{ (b, d) \}$

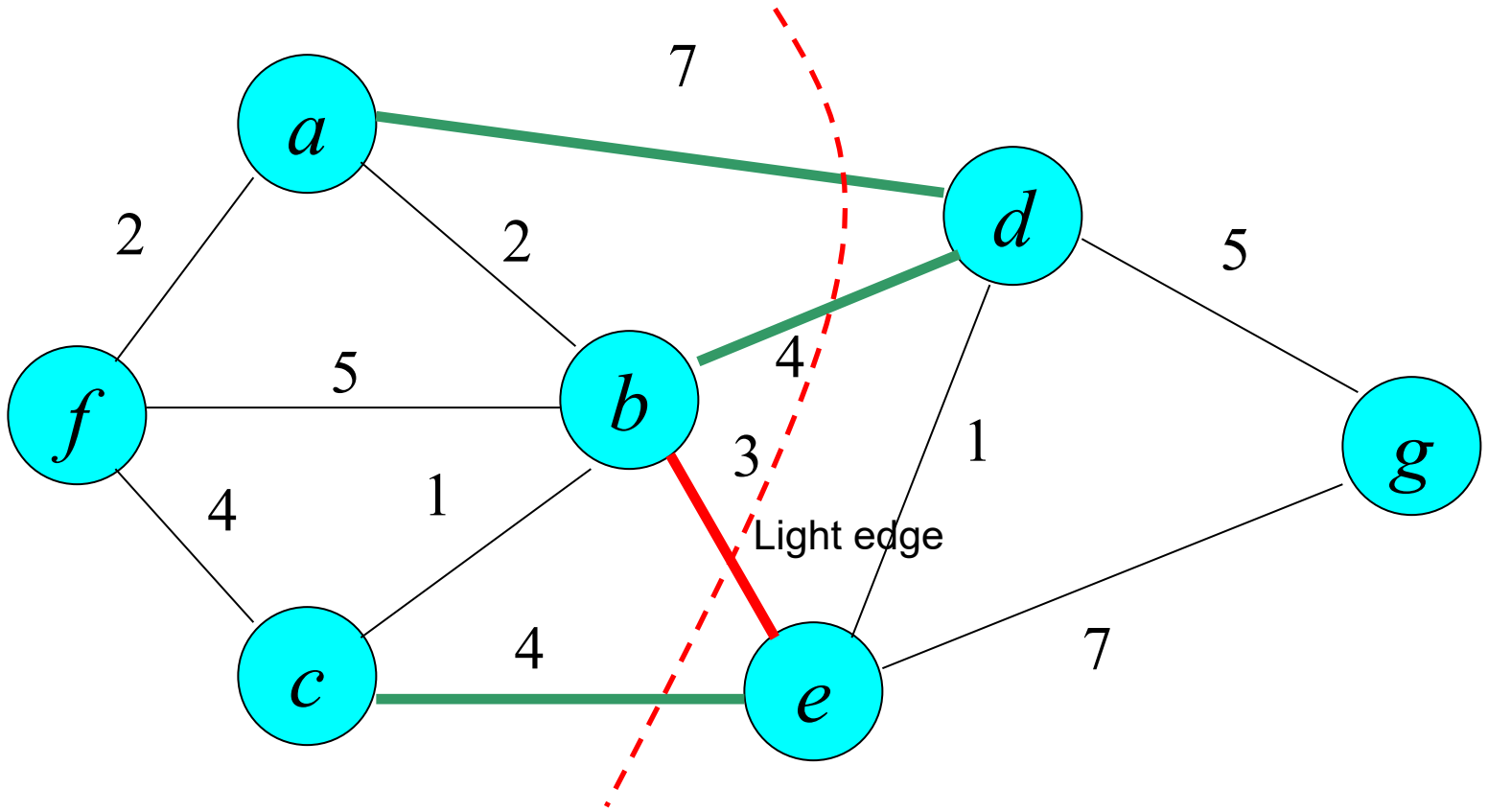


$(S, V - S)$ is compatible with A_1 and Not compatible with A_2

Light edge

- Light edge is an edge having the smallest weight among crossing edges of the cut

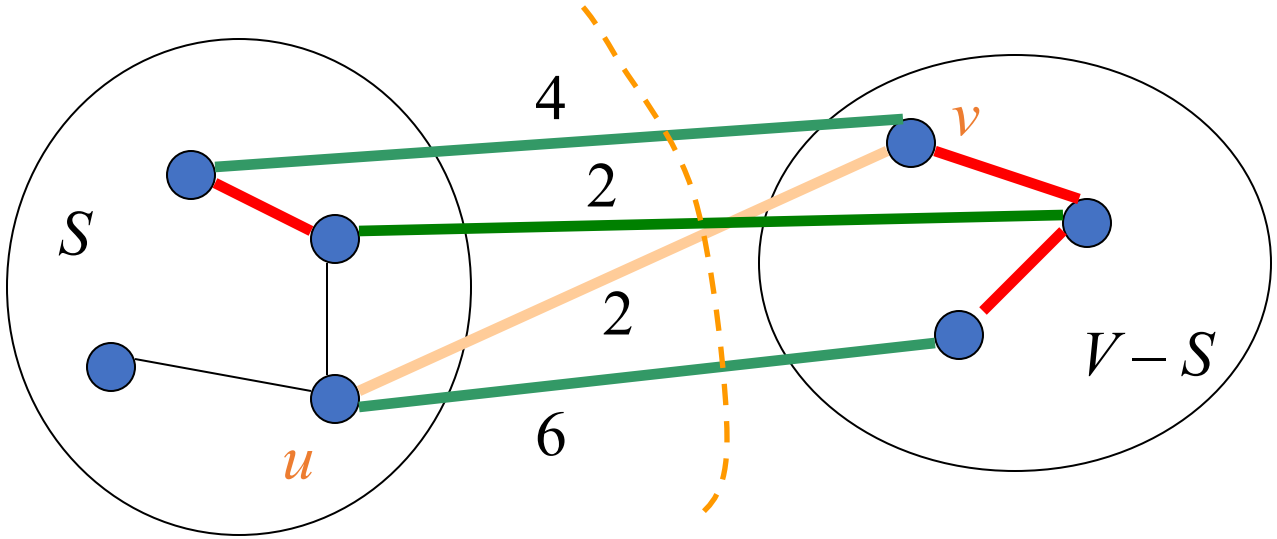
Example. Cut $(S, V\text{-}S)$: $S = \{a, b, c, f\}$



Edge (b, e) has weight 3, "smaller" than other crossing edges (a, d), (b, d), và (c, e).

Safe and light edge

- **Theorem.** Suppose $(S, V - S)$ is a cut of $G=(V, E)$ which is compatible with a subset A of E, and A is a subset of edges of a MST of G. Denote (u, v) a light and crossing edge of $(S, V - S)$. Hence, (u, v) is a safe edge of A;

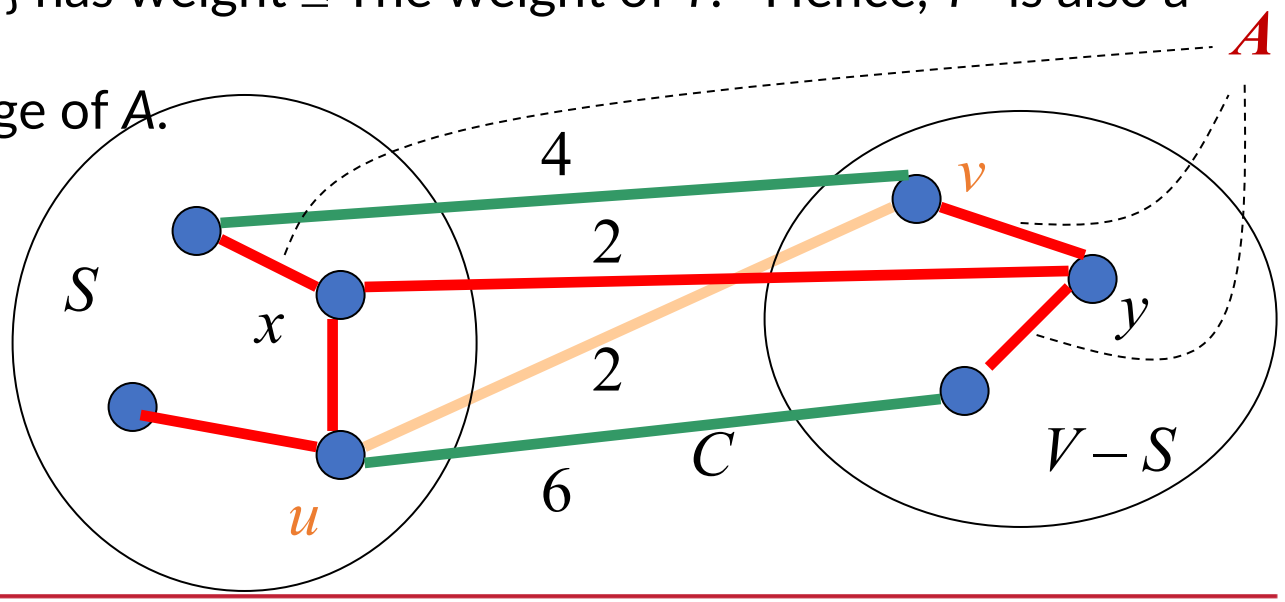


A consists of red edges.

Safe and light edge

- **We show.** (u, v) is a ***safe*** edge for A; it means: $A \cup \{(u, v)\}$ is also a subset of edges of some MST of G.
- Suppose T is a MST (red edges) containing A.

  Suppose the light edge $(u, v) \notin T$. We have
  - $T \cup \{ (u, v) \}$ contains a cycle
  - We can find an edge $(x, y) \in T$ which is a crossing edge of $(S, V - S)$.
  - The spanning tree $T' = T - \{ (x, y) \} \cup \{ (u, v) \}$ has weight $\leq$ The weight of T. Hence, T' is also a MST of G.
  - $A \cup \{ (u, v) \} \subseteq T'$, it means, (u, v) is a safe edge of A.



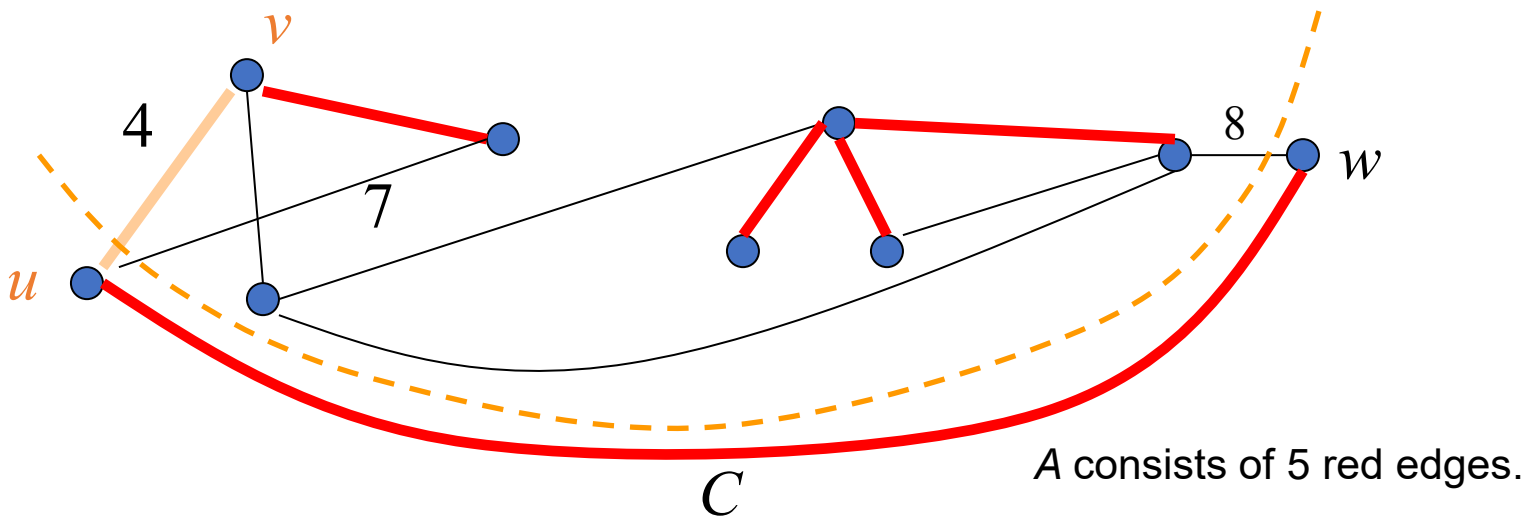
Safe and light edge

Consequence Let A be a subset of E and is a subset of the edges of some MST of $G=(V,E)$, and C is a connected component of the forest $F = (V, A)$. If (u, v) is a light edge connecting C with another connected component of F, then (u, v) is a safe edge for A.

Proof

Edge (u, v) is a light and crossing edge of the cut $(C, V - C)$ which is compatible with A.

With above theorem, edge (u, v) is safe for A.



Find safe edge

- Suppose *T* is a subset of the edges of some MST.

Kruskal Algorithm

- *T* is a forest.
- The selected safe edge for *T* is the edge having the smallest weight that connecting two connected components of *T*

Prim Algorithm

- *T* is a tree.
- The selected safe edge is the edge having the smallest weight and connecting a node of *T* and another node that is not in *T*.

Kruskal Algorithm

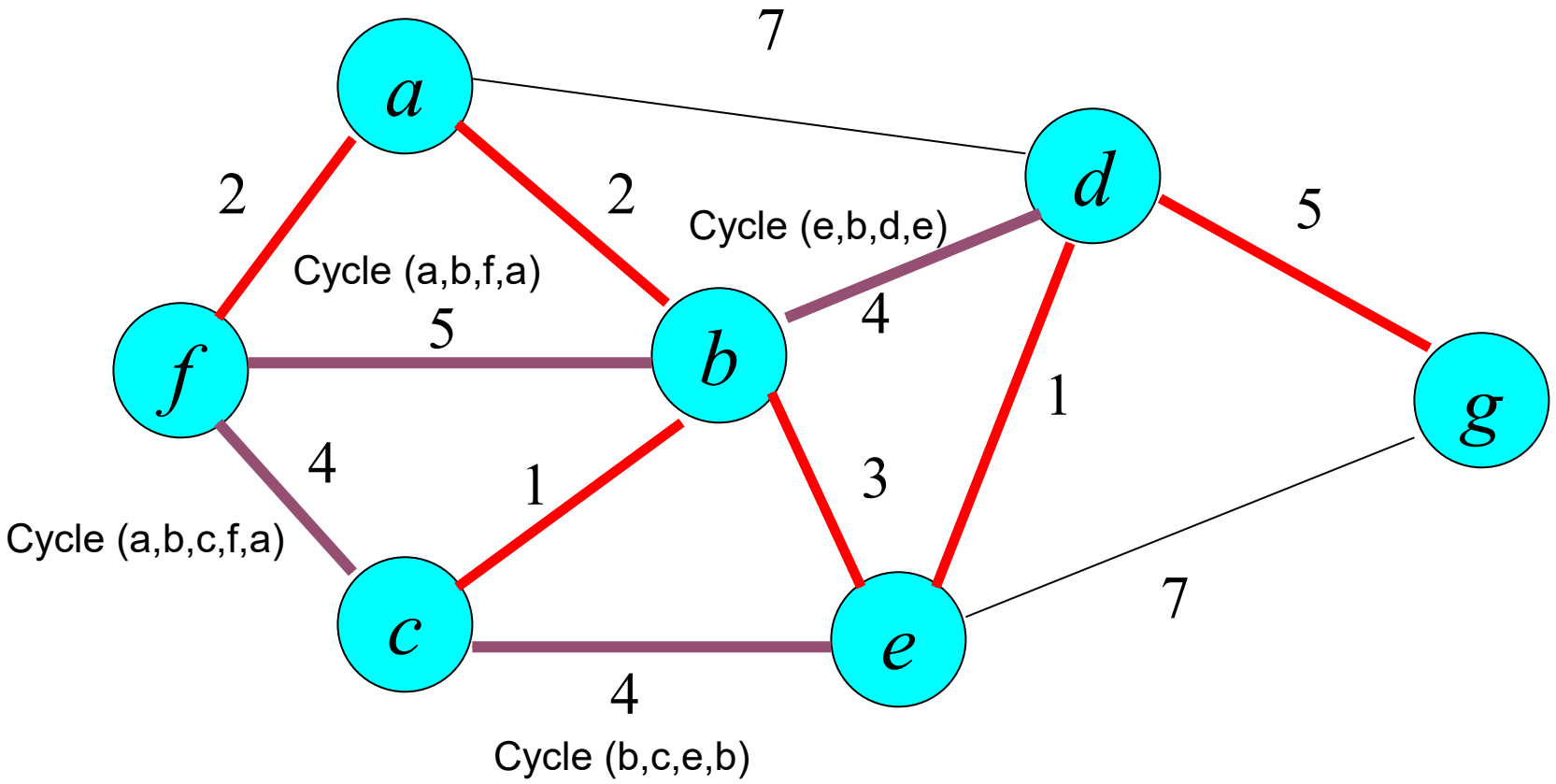
```
Generic-MST(G, c)
        T = ∅
       //Invariant: T is a subset of edges of some MST
        while T is not a spanning tree do {
            find a safe edge (u, v) for T
            T = T ∪{(u, v)}
            // T is still a subset of edges of some MST

        return T;
```

Kruskal Algorithm

- ★ *T* is a forest **(from an empty forest).**
- ★ The selected safe edge for *T* is the edge having the smallest weight that connecting two connected components of T

Kruskal Algorithm



Length of the spanning tree: 14

Kruskal Algorithm

```
Kruskal ( ){
    Sort m edges of G e_1, . . . , e_m in a non-decreasing order of weights;
    T = ∅;        // T: set of edges of the MST under construction
    for (i = 1; i <=m; i++)
         if (T ∪ {e_i} does not create any cycle) then
              T =  T ∪ {e_i};
}
```

Kruskal Algorithm

```
Kruskal ( ){
    Sort m edges of G e_1, . . . , e_m in a non-decreasing order of weights;
    T = ∅;        // T: set of edges of the MST under construction
    for (i = 1; i <=m; i++)
         if (T ∪ {e_i} does not create any cycle) then
              T =  T ∪ {e_i};
}
```

- Step 1. Sort edges.
  - Heap sort or merge sort take $O(m \log m)$
- Loop: Check if $T \cup \{ e_i \}$ contains a cycle?
  - Use DFS with time complexity of $O(m+n)$.
  - Time $O(m(m+n))$
- Total time complexity: $O(m \log m + m(m+n))$ in which n, m are number of nodes and edges of the given graph

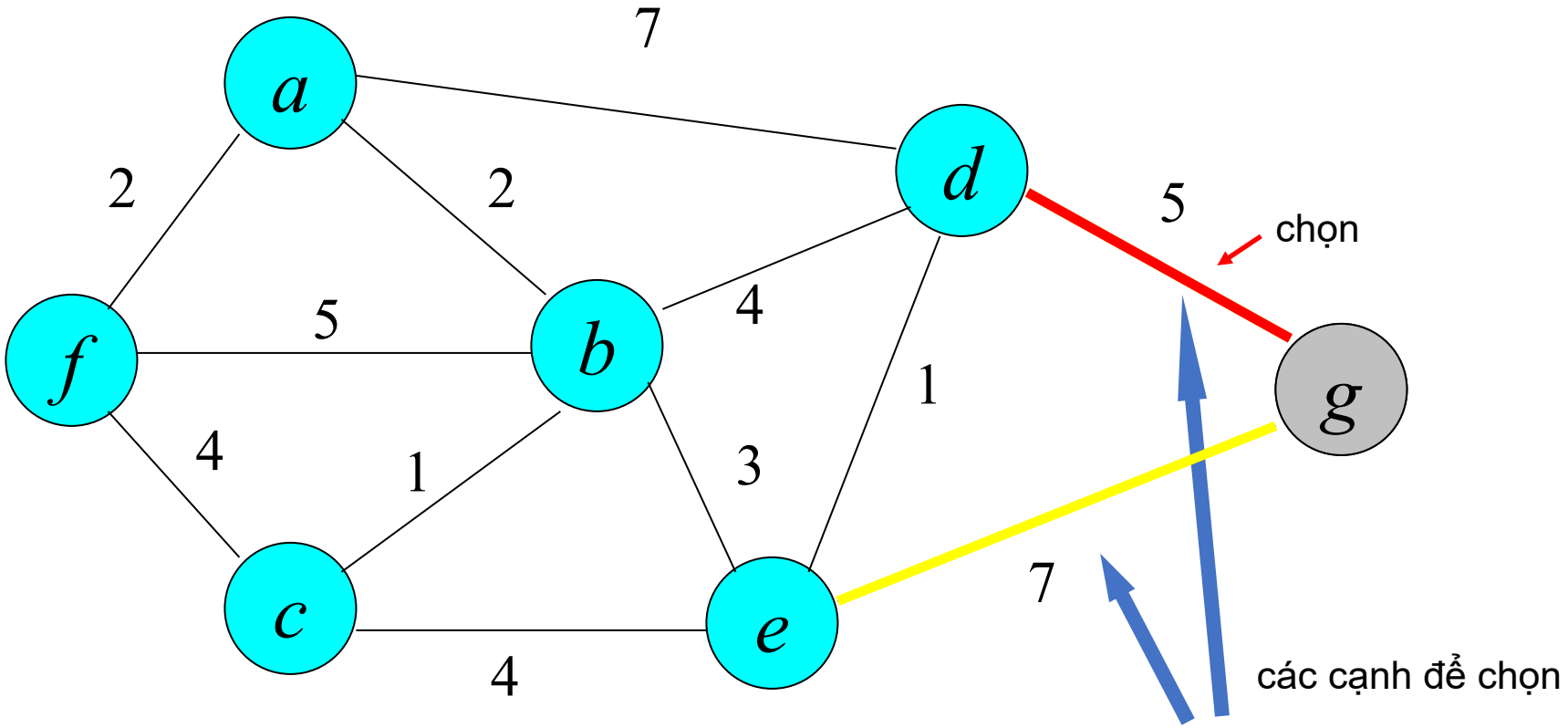
Prim Algorithm

```
Generic-MST(G, c)
       T = ∅
      //Invariant: T is a subset of edges of some MST
       while T is not a spanning tree do
           find a safe edge (u, v) for T
           T = T ∪{(u, v)}
           // T is still a subset of edges of some MST
       return T
```

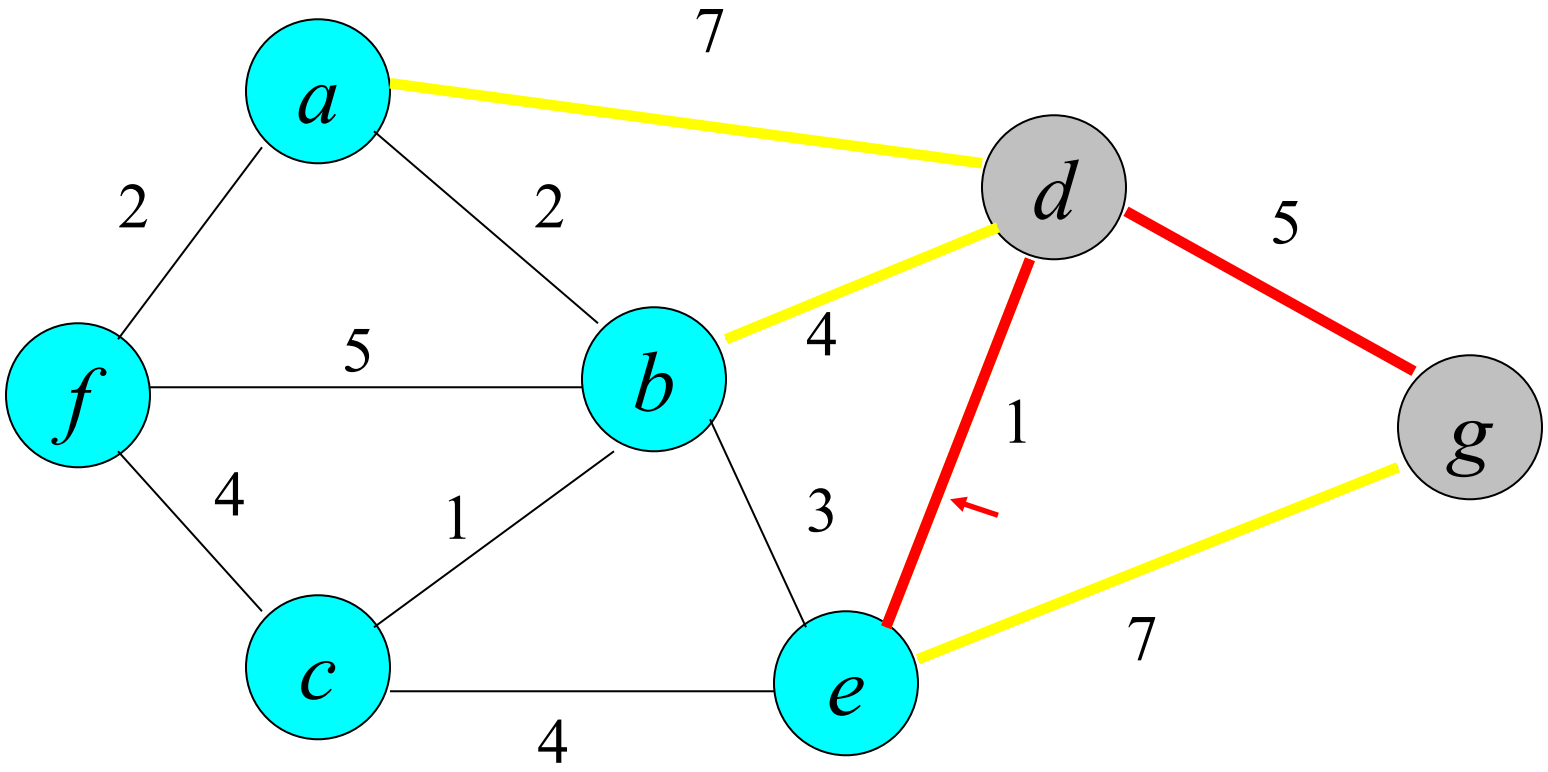
PRIM Algorithm

- ★ T is a tree (begin with a tree containing only one node)
- ★ The selected safe edge is the edge having the smallest weight and connecting a node of T and another node that is not in T.

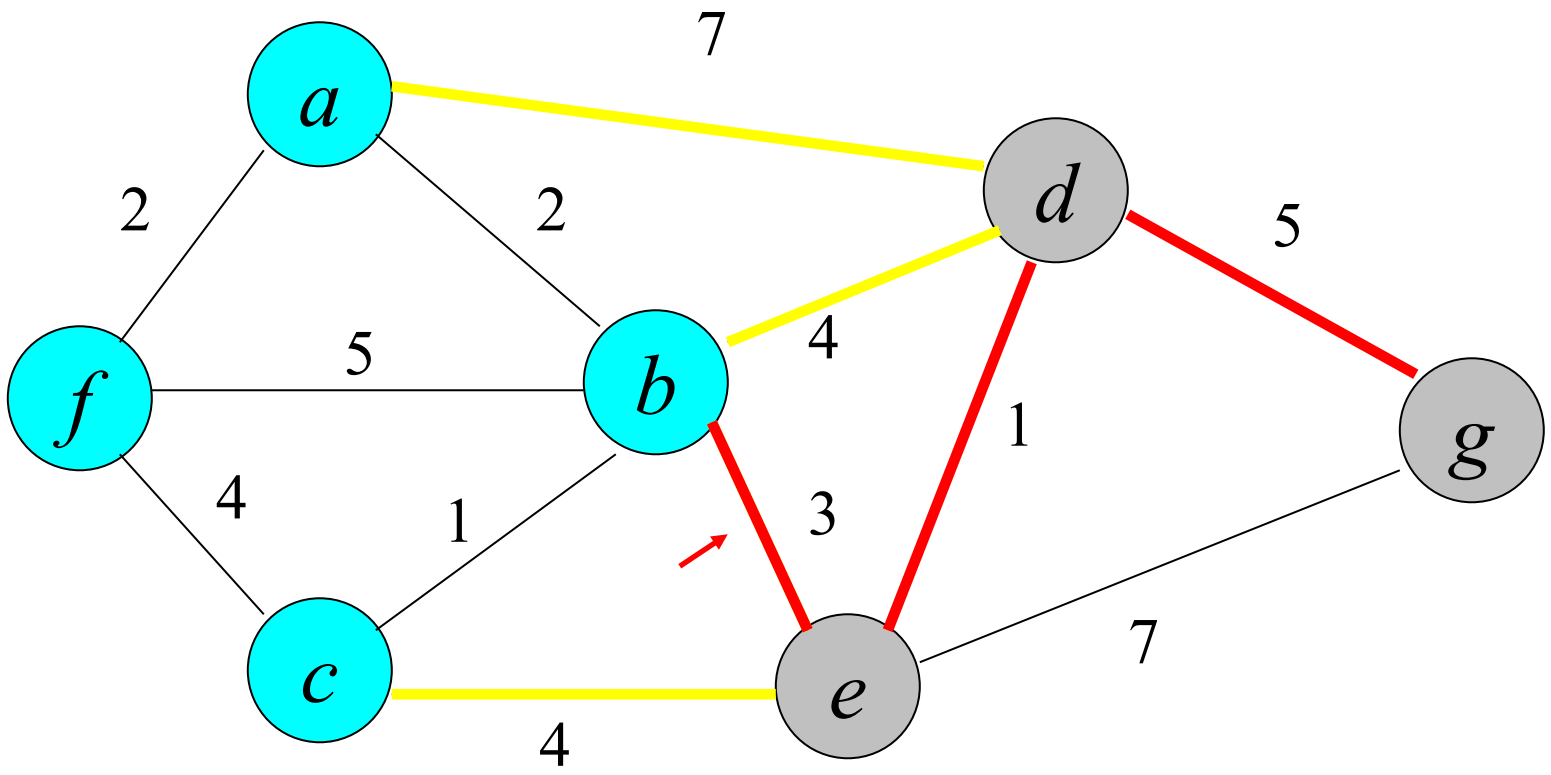
Prim Algorithm



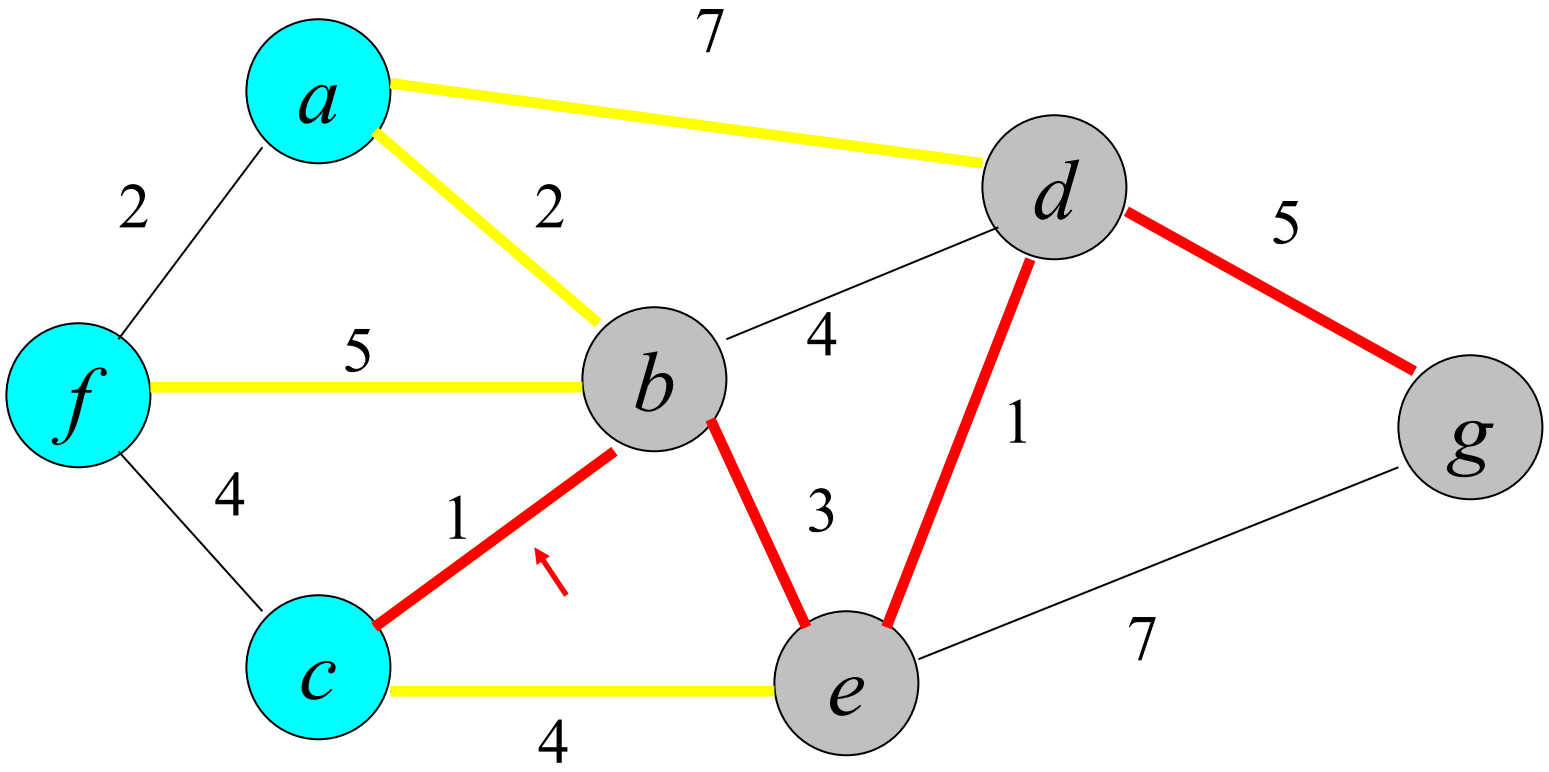
Prim Algorithm



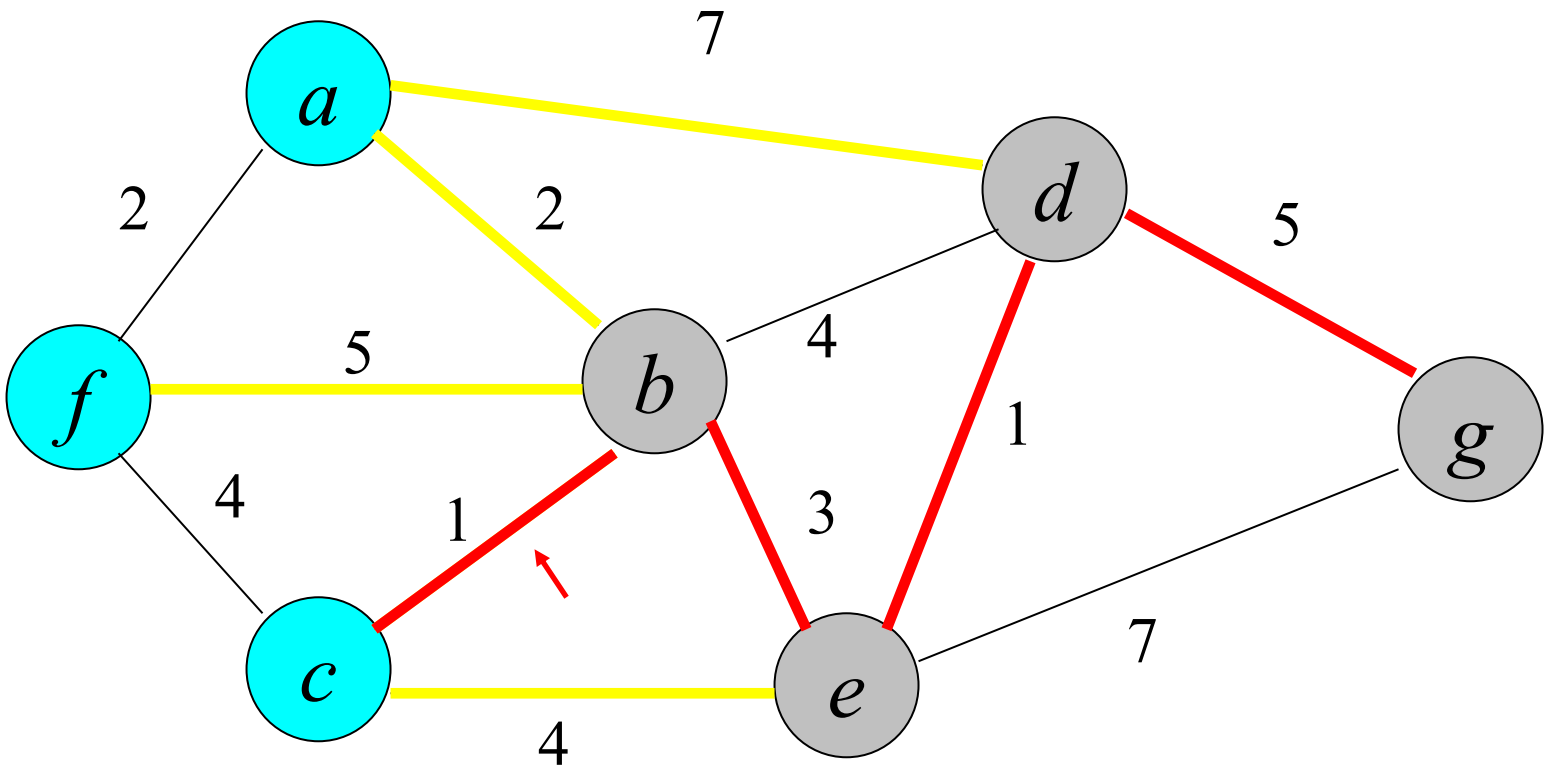
Prim Algorithm



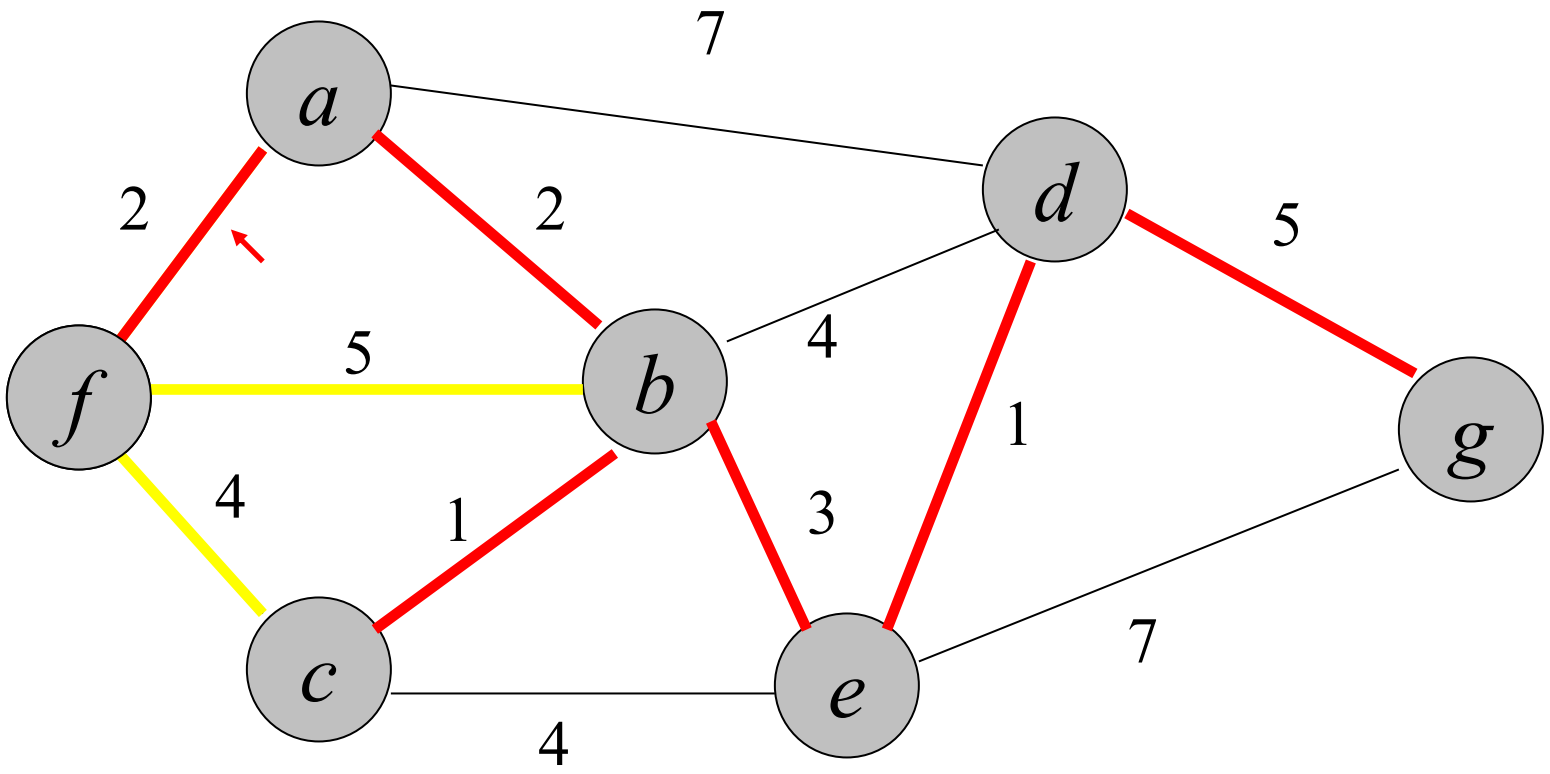
Prim Algorithm



Prim Algorithm



Prim Algorithm



CKNN gồm các cạnh: (*g*,*d*), (*d*,*e*), (*e*,*b*), (*b*,*c*), (*b*,*a*), (*a*,*f*)

Độ dài của CKNN: 14 5+1+3+1+2+2 = 14

Prim Algorithm

```
Prim(G, C){ //G: given graph; C: weight matrix
        select a node  r ∈V;
        Initialize T=(V(T), E(T)) with V(T)= {r} and E(T)=∅;
        while  V(T) < n  {
                (u, v) ← select  the smallest weight edge with u ∈ V(T) and  v∈V(G) – V(T)
                 E(T) ← E(T) ∪ { (u, v) };
                V(T) ← V(T) ∪ { v }
        }
}
```

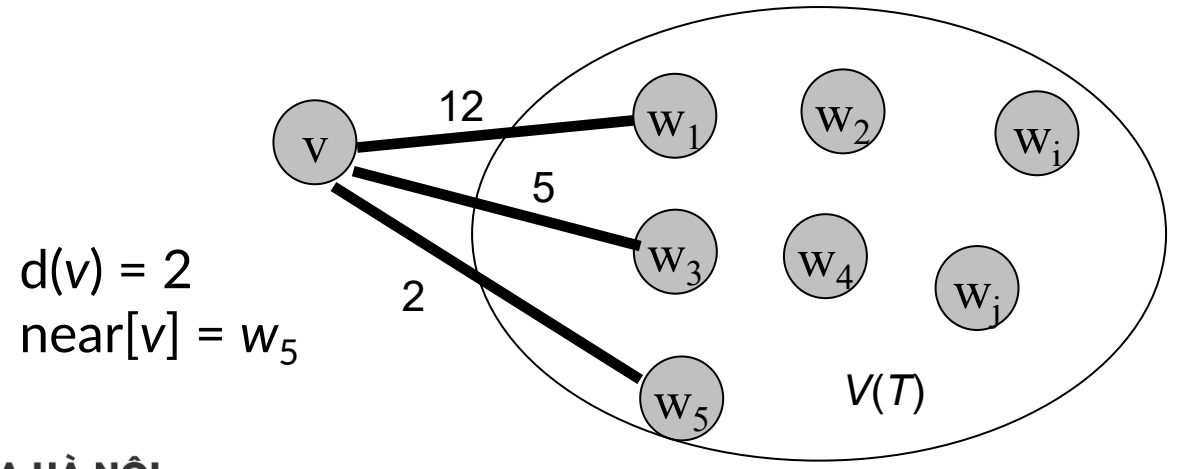
Tính đúng đắn suy từ hệ quả đã chứng minh:

Giả sử A là tập con của E và cũng là tập con của tập cạnh của CKNN của G, và C là một thành phần liên thông trong rừng $F = (V, A)$. Nếu (u, v) là cạnh nhẹ nối C với một tplt khác trong F, thì (u, v) là an toàn đối với A.

Prim Algorithm - implementation

- Suppose the graph is given by a weight matrix $C=\{c(i,j), i, j = 1, 2,..., n\}$.
- Label of a node $v \in V \setminus V(T)$ is $(d[v], near[v])$:
  - $d[v]$ distance from the node v to $V(T)$:

    $d[v] := \min\{ c(v, w) : w \in V(T) \}$
  - $near[v] := z$ the nearest node to v (c(v, z) = d[v])



d(v) = 2
near[v] = w_5

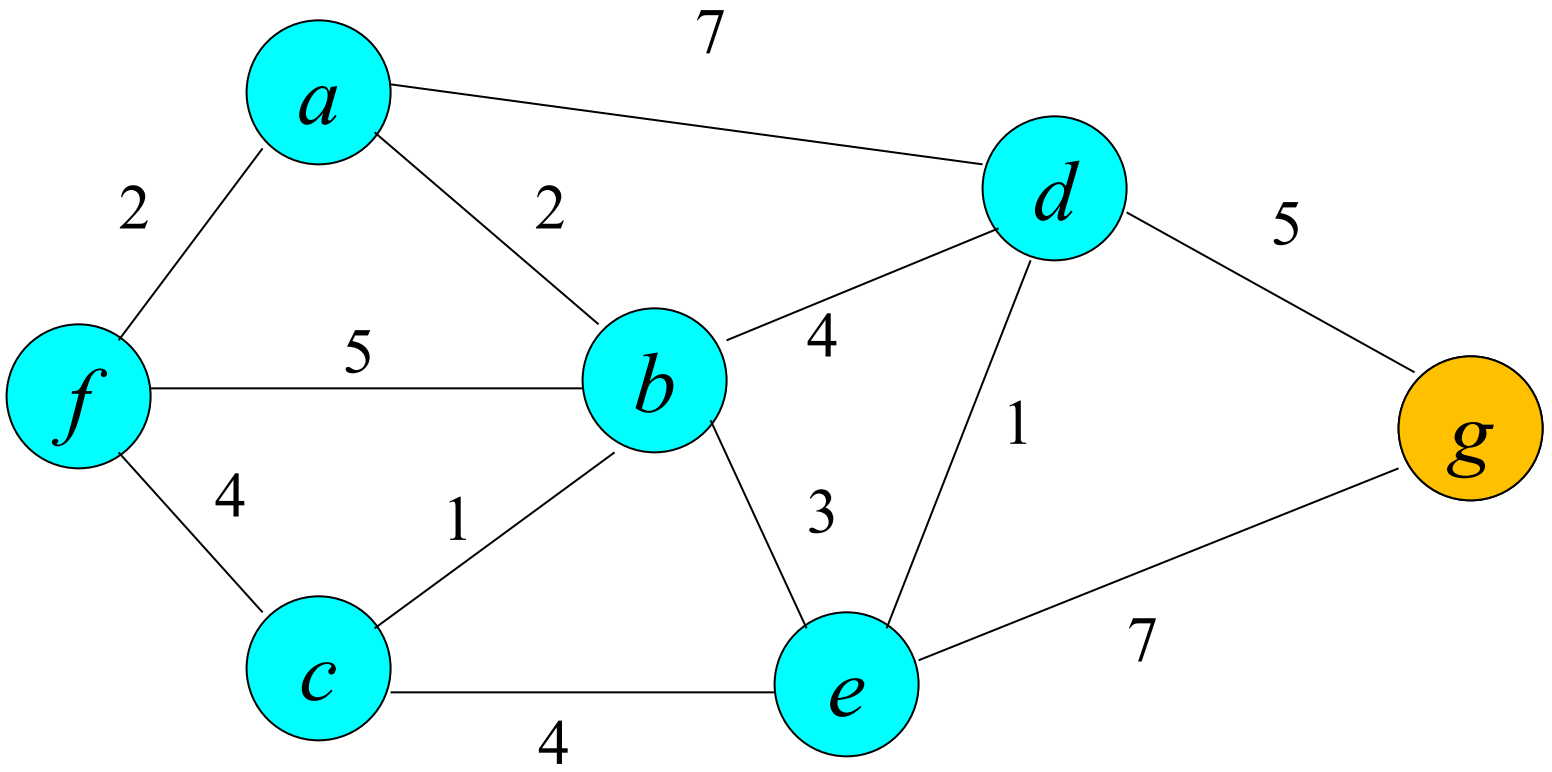
Prim Algorithm - implementation

```
Prim(G = (V, E)){
     Select r ∈ V; V(T)  = r; E(T) = ∅; d[r] = 0; near[r] = r;
     for v ∈ V \ {r} do   d[v] = c(r,v); near[v] = r;
     for k = 2 → n do {
          Select v ∈ V \ V(T) such that d[v] is minimal;
          V(T) = V(T) ∪ {v}; E(T) = E(T) ∪ {(v, near[v])};
          for u ∈ V \ V(T) do {
               if d[u] > c(v, u) then  { d[u] = c(v, u); near[u] = v;  }
          }
     }
     return (V(T), E(T));
}
```

Time complexity: $O(|V|^2)$

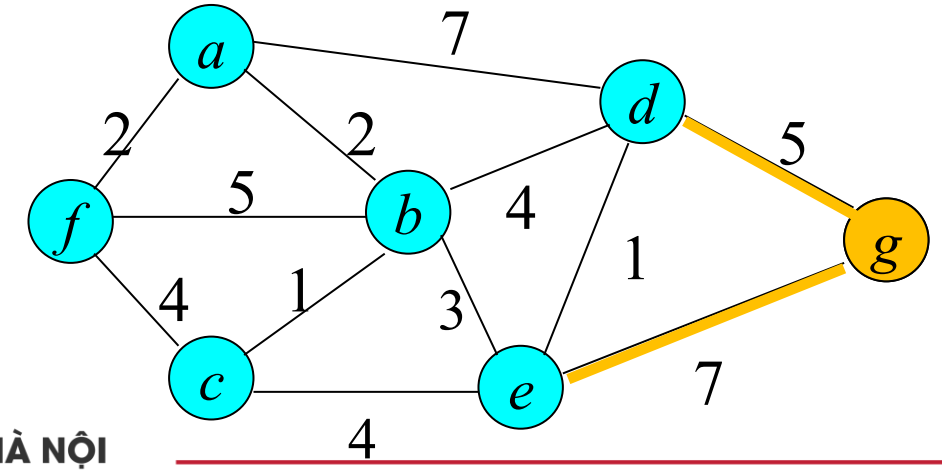
Prim Algorithm - implementation

- Example: Demonstrate the Prim algorithm for finding a minimum spanning tree of the graph below



Prim Algorithm - implementation

Node a	Node b	Node c	Node d	Node e	Node f	Node g	*V*(*T*)



HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !